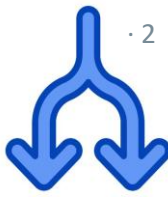


Practical Concurrent and Parallel Programming III

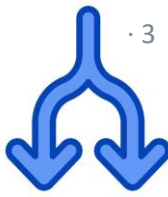
Java Memory Model

Raúl Pardo



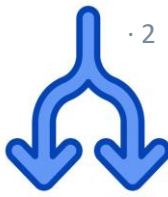
- Recall that all members of the group must be registered in the same oral feedback slot
- Contact Viktor (vihorv@itu.dk) if you have trouble finding an oral feedback slot for your group

Previously on PCPP...



- Readers and Writers Problem
- Monitors
- Fairness
- Java Intrinsic Locks (**synchronized**)
- Hardware and Programming Language Concurrency Issues
- Visibility
- Reordering
- Volatile variables (**volatile**)

Agenda

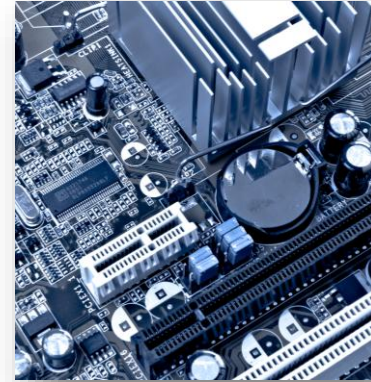


- Why do we need a memory model?
- Basic definitions
- Correctness in the Java memory model
- Visibility
- Volatile

Why do we need a memory model?



- Reasoning at the hardware (HW) level is not adequate
 - HW may be different in different computers
 - Different HW architectures have different set of instructions
 - Different HW architectures treat memory consistency differently
- Compiler and runtime environments and HW architectures may use optimizations that result in paradoxical results
 - Reordering
 - Code rewriting (e.g., branch elimination)
 - ...



Why do we need a memory model?

· 6



- Examples of paradoxical program behaviour

This program can never terminate

```
boolean running = true;
Thread t1 = new Thread(() -> {
    while (running) {
        /* do nothing */
    }
    System.out.println("t1 finishing execution");
})
t1.start();
try{Thread.sleep(500);}catch(...) {...}
running = false;
System.out.println("Main finishing execution");
```

See NoVisibility1.java

This program can output (0,0)

```
// shared variables
x=0;y=0;
a=0;b=0;

// Threads definition
Thread one = new Thread(() -> {
    a=1;
    x=b;
});
Thread other = new Thread(() -> {
    b=1;
    y=a;
});
one.start();other.start();
one.join();other.join();
System.out.println(" (" +x+" "+y+" ") );
```

See PossibleReordering.java

Why do we need a memory model?

· 6



- Examples of paradoxical program behaviour

This program can never terminate

```
boolean running = true;
Thread t1 = new Thread(() -> {
    while (running) {
        /* do nothing */
    }
    System.out.println("t1 finishing execution");
})
t1.start();
try{Thread.sleep(500);}catch(...) {...}
running = false;
System.out.println("Main finishing execution");
```

See NoVisibility1.java

How can programmers predict the behaviour of their programs in the presence of all these subtleties?

This program can output (0,0)

```
// shared variables
x=0;y=0;
a=0;b=0;

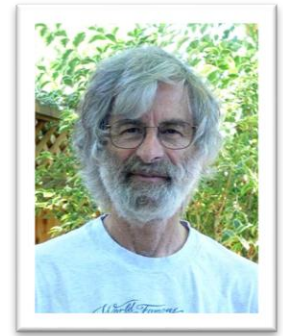
// Threads definition
Thread one = new Thread(() -> {
    a=1;
    x=b;
});
Thread other = new Thread(() -> {
    b=1;
    y=a;
});
one.start();other.start();
one.join();other.join();
System.out.println(" (" +x+" "+y+" ) );
```

See PossibleReordering.java

What is non-paradoxical behaviour?



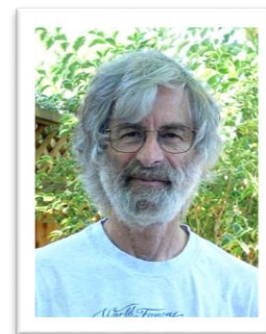
- Sequential consistency defines what programmers consider normal behaviour
- Sequential consistency was first introduced by Leslie Lamport in the late 1970s
- Intuitively, a concurrent execution is sequentially consistent iff
 - It preserves (intra-thread) program order
 - Operations appear to occur at once



What is non-paradoxical behaviour?



- Sequential consistency defines what programmers consider normal behaviour
- Sequential consistency was first introduced by Leslie Lamport in the late 1970s
- Intuitively, a concurrent execution is sequentially consistent iff
 - It preserves (intra-thread) program order
 - Operations appear to occur at once



More about sequential consistency in week 7

What is non-paradoxical behaviour?



- The execution producing the value (0,0) after executing this program is not sequentially consistent
- Ensuring sequential consistency for any program would require disabling the very same optimizations that increase performance
- The Java memory model defines for what programs it is possible to ensure sequentially consistent executions

This program can output (0,0)

```
// shared variables
x=0;y=0;
a=0;b=0;

// Threads definition
Thread one = new Thread(() -> {
    a=1;
    x=b;
});
Thread other = new Thread(() -> {
    b=1;
    y=a;
});
one.start();other.start();
one.join();other.join();
System.out.println(" (" +x+" , "+y+" ) ");
```

Why do we need a memory model?



- Java memory model
 - Set of programming language level rules that determine valid executions of a program
- The JVM is designed to only generate Java bytecode that produces executions that are valid in the memory model
 - This is enforced for all hardware and runtime environments
- This gives programmers the ability to predict the behaviour of concurrent programs

- Programming languages intended for concurrency typically have a memory model (or it is in the making)
 - Java was the first language to define its memory model
- The reasoning that we will learn today applies to any language with a well-defined memory model



Sources for logos: <https://en.wikipedia.org/w/index.php?curid=51298819>, <https://commons.wikimedia.org/w/index.php?curid=62851110>, <https://commons.wikimedia.org/w/index.php?curid=139943650>, <https://commons.wikimedia.org/w/index.php?curid=82371649>, <https://commons.wikimedia.org/w/index.php?curid=40715219>.

Java memory model: Basic definitions



An action is a pair $t(o)$ where

- t is a thread identifier
- o is an operation

There are 3 types of actions:

- Variable access
 - Read/write accesses on program variables
- Synchronization actions
 - Synchronization primitives such as acquiring and releasing a lock or monitor, starting a thread, waiting for a thread to terminate,...
- Other (any action not in the previous two categories)



- The part in yellow assigns an identifier for each operation
- As thread IDs we use t1, t2 and m, for **one**, **other**, and the main thread, respectively.
- Actions are defined by pairing the operation with the thread that executes it
 - For instance, t1(1) or m(init(a)) or t2(4).

```
// shared variables
x=0; // (init(x))
y=0; // (init(y))
a=0; // (init(a))
b=0; // (init(b))

// Threads definition
Thread one = new Thread(() -> {
    a=1; // (1)
    x=b; // (2)
});
Thread other = new Thread(() -> {
    b=1; // (3)
    y=a; // (4)
});
one.start(); // (start(t1))
other.start(); // (start(t2))
one.join(); // (join(t1))
other.join(); // (join(t2))
System.out.println("(" + x + ", " + y + ")"); // (5)
```



- The part in yellow assigns an identifier for each operation
- As thread IDs we use t1, t2 and m, for **one**, **other**, and the main thread, respectively.
- Actions are defined by pairing the operation with the thread that executes it
 - For instance, t1(1) or m(init(a)) or t2(4).

```
// shared variables
x=0; // (init(x))
y=0; // (init(y))
a=0; // (init(a))
b=0; // (init(b))

// Threads definition
Thread one = new Thread(() -> {
    a=1; // (1)
    x=b; // (2)
});
Thread other = new Thread(() -> {
    b=1; // (3)
    y=a; // (4)
});
one.start(); // (start(t1))
other.start(); // (start(t2))
one.join(); // (join(t1))
other.join(); // (join(t2))
System.out.println("(" + x + ", " + y + ")"); // (5)
```




- The part in yellow assigns an identifier for each operation
- As thread IDs we use t1, t2 and m, for **one**, **other**, and the main thread, respectively.
- Actions are defined by pairing the operation with the thread that executes it
 - For instance, t1(1) or m(init(a)) or t2(4).

```
// shared variables
x=0; // (init(x))
y=0; // (init(y))
a=0; // (init(a))
b=0; // (init(b))

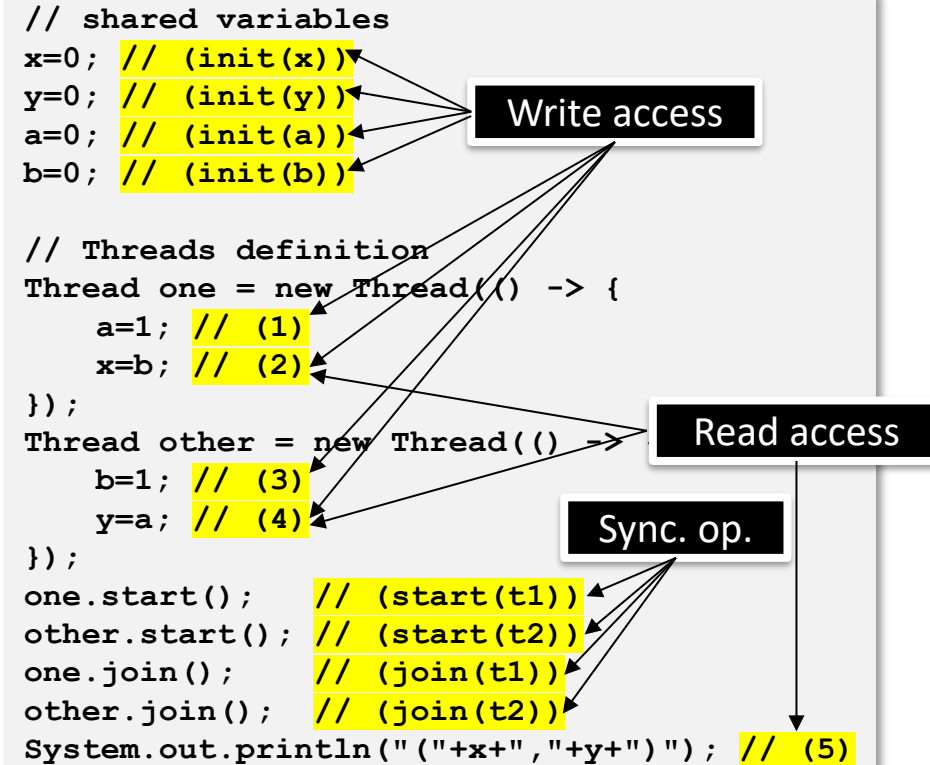
// Threads definition
Thread one = new Thread(() -> {
    a=1; // (1)
    x=b; // (2)
});
Thread other = new Thread(() -> {
    b=1; // (3)
    y=a; // (4)
});
one.start(); // (start(t1))
other.start(); // (start(t2))
one.join(); // (join(t1))
other.join(); // (join(t2))
System.out.println("(" + x + ", " + y + ")"); // (5)
```

Diagram illustrating thread actions and their access types:

- Write access:** Points to the initialization of shared variables (x, y, a, b) and the assignment of thread IDs (1) and (2) to variables a and x.
- Read access:** Points to the assignment of thread IDs (3) and (4) to variables b and y, and the final print statement (5).



- The part in yellow assigns an identifier for each operation
- As thread IDs we use t1, t2 and m, for **one**, **other**, and the main thread, respectively.
- Actions are defined by pairing the operation with the thread that executes it
 - For instance, t1(1) or m(init(a)) or t2(4).





- An execution e is a sequence of actions

$$e = a_1, a_2, a_3, \dots$$

```
// shared variables
x=0; // (init(x))
y=0; // (init(y))
a=0; // (init(a))
b=0; // (init(b))

// Threads definition
Thread one = new Thread(() -> {
    a=1; // (1)
    x=b; // (2)
});
Thread other = new Thread(() -> {
    b=1; // (3)
    y=a; // (4)
});
one.start(); // (start(t1))
other.start(); // (start(t2))
one.join(); // (join(t1))
other.join(); // (join(t2))
System.out.println("(" + x + ", " + y + ")"); // (5)
```

Executions

- An execution e is a sequence of actions

$$e = a_1, a_2, a_3, \dots$$

Can you write a possible execution for the program on the right?

```
// shared variables
x=0; // (init(x))
y=0; // (init(y))
a=0; // (init(a))
b=0; // (init(b))

// Threads definition
Thread one = new Thread(() -> {
    a=1; // (1)
    x=b; // (2)
});
Thread other = new Thread(() -> {
    b=1; // (3)
    y=a; // (4)
});
one.start(); // (start(t1))
other.start(); // (start(t2))
one.join(); // (join(t1))
other.join(); // (join(t2))
System.out.println("(" + x + ", " + y + ")"); // (5)
```



menti.com
4214 6221



- Program order defines the intra-thread order of execution of the actions of a thread
- *Given two actions a , b that are executed by the same thread t , a occurs before b according to program order iff a would be executed before b when t is executed sequentially*
- Program order is a *total order* among the actions executed by a thread
 - Recall the definition of total order: https://en.wikipedia.org/wiki/Total_order
 - Intuitively, it means that each pair of actions must be related

This property is defined at the programming language level.
Compiler or hardware level reorderings are not relevant.



- The program order for thread m is the transitive closure of this pair of actions
 - m(init(x)), m(init(y))
 - m(init(y)), m(init(a))
 - m(init(a)), m(init(b))
 - m(init(b)), m(start(t1))
 - m(start(t1)), m(start(t2))
 - m(start(t2)), m(join(t1))
 - m(join(t1)), m(join(t2))
 - m(join(t2)), m(5)
- You may write as a single tuple
 - m(init(x)), m(init(y)), m(init(a)), m(init(b)), m(start(t1)), m(start(t2)), m(join(t1)), m(join(t2)), m(5)

```
// shared variables
x=0; // (init(x))
y=0; // (init(y))
a=0; // (init(a))
b=0; // (init(b))

// Threads definition
Thread one = new Thread(() -> {
    a=1; // (1)
    x=b; // (2)
});
Thread other = new Thread(() -> {
    b=1; // (3)
    y=a; // (4)
});
one.start(); // (start(t1))
other.start(); // (start(t2))
one.join(); // (join(t1))
other.join(); // (join(t2))
System.out.println("(" + x + ", " + y + ")"); // (5)
```



- Given two actions a, b in an execution, we say that a happens-before b , denoted as $a \rightarrow b$, iff
 - *Program order rule*
 - *Monitor rule*
 - *Volatile rule*
 - *Default init rule*
 - *Final field init rule*
 - *Thread start rule*
 - *Thread termination rule*
 - *Transitive rule*
- Happens-before order is a *partial order* among the actions of an execution
 - Recall the definition of partial order: https://en.wikipedia.org/wiki/Partially_ordered_set
 - Intuition: Similar to total order, but not all pairs of elements need to be related

See accompanying lecture notes
for the definition of each rule
(we cover some in the lecture)

- Given two actions a, b in an execution, we say that a happens-before b , denoted as $a \rightarrow b$, iff
 - Program order rule
“Actions a, b are executed by the same thread, and a appears before b in program order”
 - Thread start rule
“Action a is a call to `start()` and b is the first action in the started thread”
 - Thread termination rule
“Action a is the last action executed by a thread and b is a call to `join()`, by any other thread, to wait for the thread executing a ”

Program order and thread start/termination

· 19



$a \rightarrow b \rightarrow c \rightarrow \dots$ is syntactic sugar for
 $(a,b), (b,c), (a,c), \dots$

Program order

$$HB_{p0}^m = \{m(\text{init}(x)) \rightarrow m(\text{init}(y)) \rightarrow$$
$$m(\text{init}(y)) \rightarrow m(\text{init}(a)) \rightarrow$$
$$m(\text{init}(b)) \rightarrow m(\text{start}(t1)) \rightarrow$$
$$m(\text{start}(t2)) \rightarrow m(\text{join}(t1)) \rightarrow$$
$$m(\text{join}(t2)) \rightarrow m(5)\}$$
$$HB_{p0}^{t1} = \{t1(1) \rightarrow t1(2)\}$$
$$HB_{p0}^{t2} = \{t2(3) \rightarrow t2(4)\}$$

```
// shared variables
x=0; // (init(x))
y=0; // (init(y))
a=0; // (init(a))
b=0; // (init(b))

// Threads definition
Thread one = new Thread(() -> {
    a=1; // (1)
    x=b; // (2)
});
Thread other = new Thread(() -> {
    b=1; // (3)
    y=a; // (4)
});
one.start(); // (start(t1))
other.start(); // (start(t2))
one.join(); // (join(t1))
other.join(); // (join(t2))
System.out.println("(" + x + ", " + y + ")"); // (5)
```

Program order and thread start/termination



menti.com
4214 6221

$a \rightarrow b \rightarrow c \rightarrow \dots$ is syntactic sugar for
 $(a,b), (b,c), (a,c), \dots$

Program order

$HB_{po}^m = \{m(\text{init}(x)) \rightarrow m(\text{init}(y)) \rightarrow$
 $m(\text{init}(y)) \rightarrow m(\text{init}(a)) \rightarrow$
 $\rightarrow m(\text{start}(t1)) \rightarrow$
 $\rightarrow m(\text{join}(t1)) \rightarrow$
 $\rightarrow m(5)\}$

2))

4))

What are the pairs
of actions in the
happens-before
set coming from
the thread start
rule and thread
termination rule?

```
// shared variables
x=0; // (init(x))
y=0; // (init(y))
a=0; // (init(a))
b=0; // (init(b))

// Threads definition
Thread one = new Thread(() -> {
    a=1; // (1)
    x=b; // (2)
});
Thread other = new Thread(() -> {
    b=1; // (3)
    y=a; // (4)
});
one.start(); // (start(t1))
other.start(); // (start(t2))
one.join(); // (join(t1))
other.join(); // (join(t2))
System.out.println("(" + x + ", " + y + ")"); // (5)
```

- Synchronization order is a total order among the synchronization actions of an execution
- Synchronization order must satisfy two properties
 - Consistency with mutual exclusion (locking operations must be correctly nested)
 - Consistency with happens-before order
- Multiple synchronization orders may be associated to a program
 - Due to non-determinism in the execution of synchronization operations
 - Each execution has a synchronization order associated to it



- The program only has the following synchronization order associated to it

$m(\text{start}(t1)), m(\text{start}(t2)), m(\text{join}(t1)), m(\text{join}(t2))$

```
// shared variables
x=0; // (init(x))
y=0; // (init(y))
a=0; // (init(a))
b=0; // (init(b))

// Threads definition
Thread one = new Thread(() -> {
    a=1; // (1)
    x=b; // (2)
});
Thread other = new Thread(() -> {
    b=1; // (3)
    y=a; // (4)
});
one.start(); // (start(t1))
other.start(); // (start(t2))
one.join(); // (join(t1))
other.join(); // (join(t2))
System.out.println("(" + x + ", " + y + ")"); // (5)
```

- “An execution is well-formed iff it is consistent with
 - *Program order*
 - *Happens-before order*
 - *Synchronization order*”
- Given a concurrent Java program, the JVM is guaranteed to only produce well-formed executions

Correctness in the Java memory model

- Conflicting actions define the pairs of actions that may (potentially) lead to concurrency issues

“Given actions a, b in an execution, we say that actions a, b are conflicting iff they are accesses on the same (non-volatile) variable and at least one of them is a write access”

- By definition, accesses on volatile variables are never conflicting



Conflicting in executions of this program:

- Accesses involving x
 - m(init(x)), t1(2)
 - m(init(x)), m(5)
 - t1(2), m(5)
- Accesses involving y
 - m(init(y)), t2(4)
 - m(init(y)), m(5)
 - t2(4), m(5)
- ...

```
// shared variables
x=0; // (init(x))
y=0; // (init(y))
a=0; // (init(a))
b=0; // (init(b))

// Threads definition
Thread one = new Thread(() -> {
    a=1; // (1)
    x=b; // (2)
});
Thread other = new Thread(() -> {
    b=1; // (3)
    y=a; // (4)
});
one.start(); // (start(t1))
other.start(); // (start(t2))
one.join(); // (join(t1))
other.join(); // (join(t2))
System.out.println("(" + x + ", " + y + ")"); // (5)
```




- The definition of data race in the Java memory model is slightly different than the one in lecture 1
- *“Given actions a, b in an execution, there exist a data race between a, b iff*
 - *Actions a, b are conflicting*
 - *Actions a, b are not ordered by happens-before”*

Data races (revisited)

· 27



- There is a data race between $t1(1)$ and $t2(4)$
- Happens-before subsets for this program

$HB_{po}^m = \{m(\text{init}(a)) \rightarrow m(\text{init}(b)) \rightarrow m(\text{init}(x)) \rightarrow m(\text{init}(y)) \rightarrow$
 $m(\text{start}(t1)) \rightarrow m(\text{start}(t2)) \rightarrow m(\text{join}(t1)) \rightarrow m(\text{join}(t2)) \rightarrow m(5)\}$

$HB_{po}^{t1} = \{t1(1) \rightarrow t1(2)\}$

$HB_{po}^{t2} = \{t2(3) \rightarrow t2(4)\}$

$HB_{init} = \{m(\text{start}(t1)) \rightarrow t1(1), m(\text{start}(t2)) \rightarrow t2(3)\}$

$HB_{ter} = \{t1(2) \rightarrow m(\text{join}(t1)), t2(4) \rightarrow m(\text{join}(t2))\}$

- The complete happens-before set is the transitive closure of the union of the above sets, i.e., $HB = HB_{po}^m \cup HB_{po}^{t1} \cup HB_{po}^{t2} \cup HB_{init} \cup HB_{ter}$
- Recall that HB holds for all executions as there is only one possible synchronization order
- It is easy to show that neither $t1(1) \rightarrow t2(4)$ nor $t2(4) \rightarrow t1(1)$ are not in HB , as there are no action pairs relating $t1$ and $t2$
- All the above implies that, for all executions of the program, there exist a data race between $t1(1)$ and $t2(4)$

```
// shared variables
x=0; // (init(x))
y=0; // (init(y))
a=0; // (init(a))
b=0; // (init(b))

// Threads definition
Thread one = new Thread(() -> {
    a=1; // (1)
    x=b; // (2)
});
Thread other = new Thread(() -> {
    b=1; // (3)
    y=a; // (4)
});
one.start(); // (start(t1))
other.start(); // (start(t2))
one.join(); // (join(t1))
other.join(); // (join(t2))
System.out.println("(" + x + ", " + y + ")"); // (5)
```

Data races (revisited)

Are there other data races in the executions of this program?



- There is a data race between $t1(1)$ and $t2(4)$

- Happens-before subsets for this program

$HB_{po}^m = \{m(\text{init}(a)) \rightarrow m(\text{init}(b)) \rightarrow m(\text{init}(x)) \rightarrow m(\text{init}(y)) \rightarrow m(\text{start}(t1)) \rightarrow m(\text{start}(t2)) \rightarrow m(\text{join}(t1)) \rightarrow m(\text{join}(t2)) \rightarrow m(5)\}$

$HB_{po}^{t1} = \{t1(1) \rightarrow t1(2)\}$

$HB_{po}^{t2} = \{t2(3) \rightarrow t2(4)\}$

$HB_{init} = \{m(\text{start}(t1)) \rightarrow t1(1), m(\text{start}(t2)) \rightarrow t2(3)\}$

$HB_{ter} = \{t1(2) \rightarrow m(\text{join}(t1)), t2(4) \rightarrow m(\text{join}(t2))\}$

- The complete happens-before set is the transitive closure of the union of the above sets, i.e., $HB = HB_{po}^m \cup HB_{po}^{t1} \cup HB_{po}^{t2} \cup HB_{init} \cup HB_{ter}$
- Recall that HB holds for all executions as there is only one possible synchronization order
- It is easy to show that neither $t1(1) \rightarrow t2(4)$ nor $t2(4) \rightarrow t1(1)$ are not in HB , as there are no action pairs relating $t1$ and $t2$
- All the above implies that, for all executions of the program, there exist a data race between $t1(1)$ and $t2(4)$

```
// shared variables
```

```
x=0; // (init(x))
```

```
y=0; // (init(y))
```

```
a=0; // (init(a))
```

```
b=0; // (init(b))
```

```
// Threads definition
```

```
Thread one = new Thread(()
```

```
    a=1; // (1)
```

```
    x=b; // (2)
```

```
));
```

```
Thread other = new Thread((
```

```
    b=1; // (3)
```

```
    y=a; // (4)
```

```
));
```

```
one.start(); // (start(t1))
```

```
other.start(); // (start(t2))
```

```
one.join(); // (join(t1))
```

```
other.join(); // (join(t2))
```

```
System.out.println("(" + x + ", " + y + ")"); // (5)
```



menti.com
4214 6221

- Data race freedom is the notion of correctness adopted by the Java memory model

“A program is correctly synchronized iff none of its executions contains data races”

- The JVM guarantees that all executions of correctly synchronized programs are sequentially consistent
 - No more paradoxical executions 😊



- This program is not correctly synchronized, as we found a data race in an execution
 - In fact, the data race exists in all executions, but it is sufficient to find one to argue that the program is not correctly synchronized

```
// shared variables
x=0; // (init(x))
y=0; // (init(y))
a=0; // (init(a))
b=0; // (init(b))

// Threads definition
Thread one = new Thread(() -> {
    a=1; // (1)
    x=b; // (2)
});
Thread other = new Thread(() -> {
    b=1; // (3)
    y=a; // (4)
});
one.start(); // (start(t1))
other.start(); // (start(t2))
one.join(); // (join(t1))
other.join(); // (join(t2))
System.out.println("(" + x + ", " + y + ")"); // (5)
```

Fixing the program with locks

Fixing the program with locks



- Last week we show that adding a lock removes the output (0,0)
- Now we show using the Java memory model why this fix works
- Consider the modified version of the program on right (operation IDs have also changed)
- We removed the join() and print() statements for simplicity and w.l.o.g.
 - There are no data races between the last writes on x and y in t1 and t2 and the printing statement in m, since these accesses were ordered by the thread termination rules

```
// shared variables
l = new ReentrantLock(); // (init(l))
x=0; // (init(x))
y=0; // (init(y))
a=0; // (init(a))
b=0; // (init(b))

// Threads definition
Thread one = new Thread() -> {
    l.lock(); // (1)
    a=1; // (2)
    x=b; // (3)
    l.unlock(); // (4)
};
Thread other = new Thread() -> {
    l.lock() // (5)
    b=1; // (6)
    y=a; // (7)
    l.unlock(); // (8)
};
one.start(); // (start(t1))
other.start(); // (start(t2))
```



- Given two actions a, b in an execution, we say that a happens-before b , denoted as $a \rightarrow b$, iff
 - Monitor rule
 - “Action a is an `unlock()` operation on a monitor, and b is a subsequent `lock()` operation on the same monitor”*

“Monitor” refers to any synchronization primitive
(lock, conditions, semaphore, barrier, ...)

“`unlock()/lock()`” refers to acquiring/releasing the
corresponding synchronization primitive

Fixing the program with locks

· 33



- HB_{po}^m , HB_{po}^{t1} , HB_{po}^{t2} and HB_{init} are the same as before
 - Just including the program order pairs for the new actions

```
// shared variables
l = new ReentrantLock(); // (init(l))
x=0; // (init(x))
y=0; // (init(y))
a=0; // (init(a))
b=0; // (init(b))

// Threads definition
Thread one = new Thread() -> {
    l.lock(); // (1)
    a=1; // (2)
    x=b; // (3)
    l.unlock(); // (4)
});
Thread other = new Thread() -> {
    l.lock() // (5)
    b=1; // (6)
    y=a; // (7)
    l.unlock(); // (8)
});
one.start(); // (start(t1))
other.start(); // (start(t2))
```

Fixing the program with locks



- Adding locks introduced four new synchronization actions

t1(1), t1(4), t2(5), t2(8)

- Now we have more than one synchronization order

(m(start(t1)), t1(1), t1(4), m(start(t2)), t2(5), t2(8))

(m(start(t1)), t1(1), m(start(t2)), t1(4), t2(5), t2(8))

(m(start(t1)), m(start(t2)), t1(1), t1(4), t2(5), t2(8))

(m(start(t1)), m(start(t2)), t2(5), t2(8), t1(1), t1(4))

```
// shared variables
l = new ReentrantLock(); // (init(l))
x=0; // (init(x))
y=0; // (init(y))
a=0; // (init(a))
b=0; // (init(b))

// Threads definition
Thread one = new Thread() -> {
    l.lock(); // (1)
    a=1; // (2)
    x=b; // (3)
    l.unlock(); // (4)
};
Thread other = new Thread() -> {
    l.lock() // (5)
    b=1; // (6)
    y=a; // (7)
    l.unlock(); // (8)
};
one.start(); // (start(t1))
other.start(); // (start(t2))
```

Fixing the program with locks



- Adding locks introduced four new synchronization actions
t1(1), t1(4), t2(5), t2(8)
- Now we have more than one synchronization order

(m(start(t1)), t1(1), **t1(4)**, m(start(t2)), **t2(5)**, t2(8))

(m(start(t1)), t1(1), m(start(t2)), **t1(4)**, **t2(5)**, t2(8))

(m(start(t1)), m(start(t2)), t1(1), **t1(4)**, **t2(5)**, t2(8))

(m(start(t1)), m(start(t2)), t2(5), **t2(8)**, **t1(1)**, t1(4))

```
// shared variables  
l = new ReentrantLock(); // (init(1))
```

t1 acquires the lock first
t1(4) -> t2(5) holds

```
// Threads definition  
Thread one = new Thread(() -> {  
    l.lock(); // (1)  
    a=1; // (2)  
    x=b; // (3)  
    l.unlock(); // (4)  
});  
Thread other = new Thread(() -> {  
    l.lock() // (5)  
    b=1; // (6)  
    y=a; // (7)  
    l.unlock(); // (8)  
});
```

t2 acquires the lock first
t2(8) -> t1(1) holds

Fixing the program with locks



- Based on program order and synchronization orders, there are two happens-before order sets that characterize the order of executions of the program

$HB^1 = HB \cup \{t1(4) \rightarrow t2(5)\}$

$HB^2 = HB \cup \{t2(8) \rightarrow t1(1)\}$

- The action pairs order by happens-before in the executions of this program are given by the transitive closure of HB^1 or HB^2 ; depending on what thread acquires the lock first

```
// shared variables  
l = new ReentrantLock(); // (init(l))  
x=0; // (init(x))  
y=0; // (init(y))
```

t1 acquires the lock first
t1(4) → t2(5) holds

```
Thread one = new Thread(() -> {  
    l.lock(); // (1)  
    a=1; // (2)  
    x=b; // (3)  
    l.unlock(); // (4)  
});  
Thread other = new Thread(() -> {  
    l.lock() // (5)  
    b=1; // (6)  
    y=a; // (7)  
    l.unlock(); // (8)  
});
```

t2 acquires the lock first
t2(8) → t1(1) holds

Fixing the program with locks

· 37



- Now we can show that the data race between $t1(2)$ and $t2(7)$ does not exist in any execution
- We must show that either $t1(2) \rightarrow t2(7)$ or $t2(7) \rightarrow t1(2)$ in HB^1 and HB^2
- In HB^1 we have

$t1(2) \xrightarrow{(po)} t1(3) \xrightarrow{(po)} t1(4) \xrightarrow{(sync)} t2(5) \xrightarrow{(po)} t2(6) \xrightarrow{(po)} t2(7)$

- In HB^2 we have

$t2(7) \xrightarrow{(po)} t2(8) \xrightarrow{(sync)} t1(1) \xrightarrow{(po)} t1(2)$

```
// shared variables
l = new ReentrantLock(); // (init(l))
x=0; // (init(x))
y=0; // (init(y))
a=0; // (init(a))
b=0; // (init(b))

// Threads definition
Thread one = new Thread(() -> {
    l.lock(); // (1)
    a=1; // (2)
    x=b; // (3)
    l.unlock(); // (4)
});
Thread other = new Thread(() -> {
    l.lock() // (5)
    b=1; // (6)
    y=a; // (7)
    l.unlock(); // (8)
});
one.start(); // (start(t1))
other.start(); // (start(t2))
```

Fixing the program with locks

· 37



- Now we can show that the data race between $t1(2)$ and $t2(7)$ does not exist in any execution
- We must show that either $t1(2) \rightarrow t2(7)$ or $t2(7) \rightarrow t1(2)$ in HB^1 and HB^2
- In HB^1 we have

$t1(2) \xrightarrow{(po)} t1(3) \xrightarrow{(po)} t1(4) \xrightarrow{(sync)} t2(5) \xrightarrow{(po)} t2(6) \xrightarrow{(po)} t2(7)$

- In HB^2 we have

$t2(7) \xrightarrow{(po)} t2(8) \xrightarrow{(sync)} t1(1) \xrightarrow{(po)} t1(2)$

The same holds
for the other pairs
of conflicting
actions

```
// shared variables
l = new ReentrantLock(); // (init(l))
x=0; // (init(x))
y=0; // (init(y))
a=0; // (init(a))
b=0; // (init(b))

// Threads definition
Thread one = new Thread(() -> {
    l.lock(); // (1)
    a=1; // (2)
    x=b; // (3)
    l.unlock(); // (4)
});
Thread other = new Thread(() -> {
    l.lock() // (5)
    b=1; // (6)
    y=a; // (7)
    l.unlock(); // (8)
});
one.start(); // (start(t1))
other.start(); // (start(t2))
```

Fixing the program with locks

- Now we can show that the data race between $t1(2)$ and $t2(7)$ does not exist in any execution
- We must show that either $t1(2) \rightarrow t2(7)$ or $t2(7) \rightarrow t1(2)$ in HB^1 and HB^2
- In HB^1 we have

$t1(2) \rightarrow t1(3) \rightarrow t1(4) \rightarrow t2(5) \rightarrow t2(6) \rightarrow t2(7)$
(po) (po) (sync) (po) (po)

- In HB^2 we have

$t2(7) \rightarrow t2(8) \rightarrow t1(1) \rightarrow t1(2)$
(po) (sync) (po)

The same holds
for the other pairs
of conflicting
actions

Is this program
correctly synchronized?



```
// shared variables
l = new ReentrantLock();
x=0; // (init(x))
y=0; // (init(y))
a=0; // (init(a))
b=0; // (init(b))
```

```
// Threads definition
Thread one = new Thread() {
    1.lock(); // (1)
    a=1; // (2)
    x=b; // (3)
    1.unlock(); // (4)
};
```

```
Thread other = new Thread() -> {
    1.lock() // (5)
    b=1; // (6)
    y=a; // (7)
    1.unlock(); // (8)
};
one.start(); // (start(t1))
other.start(); // (start(t2))
```



menti.com
4214 6221

Visibility in the Java memory model

- Given two actions a, b in an execution, if it holds that $a \rightarrow b$, then the effect of a is visible by b
- In other words, we simply need to ensure that actions are ordered by happens-before to ensure visibility
- Using this property we can also predict the value that a read access on a variable will read



- We want to show that x equals 1 is a possible output of this program. If the following property holds in HB, then x equals 1 after running the program
 $m(\text{init}(x)) \rightarrow m(\text{init}(b)) \rightarrow t2(6) \rightarrow t1(3)$
- This ensures that $t2(6)$ is the last write on x in the program, and that $t2(6)$ is the last write on b , so $t1(3)$ must write a 1 on x
- We start checking that $m(\text{init}(x)) \rightarrow m(\text{init}(b)) \rightarrow t2(6)$ is in HB
- $m(\text{init}(x)) \rightarrow m(\text{init}(b)) \rightarrow m(\text{start}(t2)) \rightarrow t2(5) \rightarrow t2(6)$
 $\quad \quad \quad (\text{po}) \quad \quad \quad (\text{po}) \quad \quad \quad (\text{init}) \quad \quad (\text{po})$
 - Note that this holds for all executions, as it does not depend on synchronization order
- Now we show $t2(6) \rightarrow t1(3)$
- $t2(6) \rightarrow t2(8) \rightarrow t1(1) \rightarrow t1(3)$
 $\quad \quad \quad (\text{po}) \quad (\text{sync, HB2}) \quad (\text{po})$
 - This only holds for HB2, so only when $t2$ takes the lock first, we have that x equals 1

```
// shared variables
l = new ReentrantLock(); // (init(1))
x=0; // (init(x))
y=0; // (init(y))
a=0; // (init(a))
b=0; // (init(b))

// Threads definition
Thread one = new Thread(() -> {
    l.lock(); // (1)
    a=1; // (2)
    x=b; // (3)
    l.unlock(); // (4)
});
Thread other = new Thread(() -> {
    l.lock() // (5)
    b=1; // (6)
    y=a; // (7)
    l.unlock(); // (8)
});
one.start(); // (start(t1))
other.start(); // (start(t2))
```



- We want to show that x equals 1 is a possible output of this program. If the following property holds in HB, then x equals 1 after running the program
 $m(\text{init}(x)) \rightarrow m(\text{init}(b)) \rightarrow t2(6) \rightarrow t1(3)$
- This ensures that $t2(6)$ is the last write on x in the program, and that $t2(6)$ is the last write on b , so $t1(3)$ must write a 1 on x
- We start checking that $m(\text{init}(x)) \rightarrow m(\text{init}(b)) \rightarrow t2(6)$ is in HB
- $m(\text{init}(x)) \rightarrow m(\text{init}(b)) \rightarrow m(\text{start}(t2)) \rightarrow t2(5) \rightarrow t2(6)$
 $\quad \quad \quad (\text{po}) \quad \quad \quad (\text{po}) \quad \quad \quad (\text{init}) \quad \quad \quad (\text{po})$
 - Note that this holds for all executions, as it does not depend on synchronization order
- Now we show $t2(6) \rightarrow t1(3)$
- $t2(6) \rightarrow t2(8) \rightarrow t1(1) \rightarrow t1(3)$
 $\quad \quad \quad (\text{po}) \quad \quad (\text{sync, HB2}) \quad \quad (\text{po})$
 - This only holds for HB2, so only when $t2$ takes the lock first, we have that x equals 1

```
// shared variables
l = new ReentrantLock(); // (init(l))
x=0; // (init(x))
y=0; // (init(y))
a=0; // (init(a))
b=0; // (init(b))

// Threads definition
Thread one = new Thread(() -> {
    l.lock(); // (1)
    a=1; // (2)
    x=b; // (3)
    l.unlock(); // (4)
});
Thread other = new Thread(() -> {
    l.lock() // (5)
    b=1; // (6)
    y=a; // (7)
    l.unlock(); // (8)
});
one.start(); // (start(t1))
other.start(); // (start(t2))
```

Volatile variables in the Java memory model

- Given two actions a, b in an execution, we say that a happens-before b , denoted as $a \rightarrow b$, iff
 - Volatile rule
“Action a is a write access on a volatile variable, and b is a subsequent read access on the same volatile variable.”
- Volatile variables introduce more possible synchronization orders (as they are synchronization actions but do not ensure mutual exclusion)
- Volatile variables reduce the number of conflicting actions, as, by definition, accesses on volatile variables are not conflicting



- Consider the following fix for the program on the right, using volatile variables
- We can easily conclude that this program is correctly synchronized
- Only accesses to x, y are conflicting, i.e.,
 - m(init(x)), t1(2)
 - m(init(y)), t2(4)

We have already seen that these accesses are ordered by happens-before via the thread start rule

```
// shared variables
x=0; // (init(x))
y=0; // (init(y))
volatile a=0; // (init(a))
volatile b=0; // (init(b))

// Threads definition
Thread one = new Thread(() -> {
    a=1; // (1)
    x=b; // (2)
});
Thread other = new Thread(() -> {
    b=1; // (3)
    y=a; // (4)
});
one.start(); // (start(t1))
other.start(); // (start(t2))
```



- Volatile variables have notably incremented the number of synchronization orders
- Here we list a few depending only on the order of accesses on volatile variables
 - ...,t1(1),...,t1(2),...,t2(3),...,t2(4),...
 - ...,t2(3),...,t2(4),...,t1(1),...,t1(2),...
 - ...,t1(1),...,t2(3),...,t1(2),...,t2(4),...
 - ...,t2(3),...,t1(1),...,t1(2),...,t2(4),...
 - ...,t1(1),...,t2(3),...,t2(4),...,t1(2),...
 - ...,t2(3),...,t1(1),...,t2(4),...,t1(2),...
 - ...

```
// shared variables
x=0; // (init(x))
y=0; // (init(y))
volatile a=0; // (init(a))
volatile b=0; // (init(b))

// Threads definition
Thread one = new Thread(() -> {
    a=1; // (1)
    x=b; // (2)
});
Thread other = new Thread(() -> {
    b=1; // (3)
    y=a; // (4)
});
one.start(); // (start(t1))
other.start(); // (start(t2))
```



- Volatile variables have notably incremented the number of synchronization orders
- Here we list a few depending only on the order of accesses on volatile variables
 - ...,t1(1),...,t1(2),...,t2(3),...,t2(4),...
 - ...,t2(3),...,t2(4),...,t1(1),...,t1(2),...
 - ...,t1(1),...,t2(3),...,t1(2),...,t2(4),...
 - ...,t2(3),...,t1(1),...,t1(2),...,t2(4),...
 - ...,t1(1),...,t2(3),...,t2(4),...,t1(2),...
 - ...,t2(3),...,t1(1),...,t2(4),...,t1(2),...
 - ...

```
// shared variables
x=0; // (init(x))
y=0; // (init(y))
volatile a=0; // (init(a))
volatile b=0; // (init(b))

// Threads definition
Thread one = new Thread(() -> {
    a=1; // (1)
    x=b; // (2)
});
Thread other = new Thread(() -> {
    b=1; // (3)
    y=a; // (4)
});
one.start(); // (start(t1))
other.start(); // (start(t2))
```




- Volatile variables have notably incremented the number of

Only t1(1) -> t2(4) holds

- on the order of accesses on volatile variables

- ...,t1(1),...,t1(2),...,t2(3),...,t2(4),...

- ...,t2(3),...,t2(4),...,t1(1),...,t1(2),...

- ...,t1(1),...,t2(3),...,t1(2),...,t2(4),...

- ...,t2(3),...,t1(1),...,t1(2),...,t2(4),...

- ...,t1(1),...,t2(3),...,t2(4),...,t1(2),...

- ...,t2(3),...,t1(1),...,t2(4),...,t1(2),...

- ...

Only t2(3) -> t1(2) holds

```
// shared variables
x=0; // (init(x))
y=0; // (init(y))
volatile a=0; // (init(a))
volatile b=0; // (init(b))

// Threads definition
Thread one = new Thread(() -> {
    a=1; // (1)
    x=b; // (2)
});
Thread other = new Thread(() -> {
    b=1; // (3)
    y=a; // (4)
});
one.start(); // (start(t1))
other.start(); // (start(t2))
```

Both t1(1) -> t2(4) and t2(3) -> t1(2) hold



- Volatile variables have notably incremented **(0,1)** number of
- **Only t1(1) -> t2(4) holds** on the order of accesses on volatile variables

- ...,t1(1),...,t1(2),...,t2(3),...,t2(4),...
- ...,t2(3),...,t2(4),...,t1(1),...,t1(2),...
- ...,t1(1),...,t2(3),...,t1(2),...,t2(4),...
- ...,t2(3),...,t1(1),...,t1(2),...,t2(4),...
- ...,t1(1),...,t2(3),...,t2(4),...,t1(2),...
- ...,t2(3),...,t1(1),...,t2(4),...,t1(2),...
- ...

(1,0)
Only t2(3) -> t1(2) holds

```
// shared variables
x=0; // (init(x))
y=0; // (init(y))
volatile a=0; // (init(a))
volatile b=0; // (init(b))

// Threads definition
Thread one = new Thread(() -> {
    a=1; // (1)
    x=b; // (2)
});
Thread other = new Thread(() -> {
    b=1; // (3)
    y=a; // (4)
});
one.start(); // (start(t1))
other.start(); // (start(t2))
```

(1,1)

Both t1(1) -> t2(4) and t2(3) -> t1(2) hold

Possible strategy to determine the happens-before order sets for all executions

1. Identify actions and categorize them
2. Define action pairs from the program order rule (HB_{po} in the previous examples)
3. Define action pairs from synchronization actions (i.e., actions relevant for the pairs in the sets HB_{init} , HB_{ter} , monitor rule and volatile rule in the previous examples)
4. Define the possible synchronization orders
5. Determine differences in happens-before pairs in the different synchronization orders (recall HB^1 , HB^2 in the lock example)
6. For each of the differences found in 5 define the happens-before order set associated to the executions corresponding to each synchronization order

Use the happens-before sets identified in 6 to

- Find data races
- Show absence of data races and correct synchronization
- Predict the value of variables in the executions of the program

Programmers care about data races



· 49

- Being able to find data races in your code is very important
- Programmers of popular code bases care about data races in their code; even if they are not easily detectable by tests (more about this in week 5)
- Example: <https://github.com/grpc/grpc-java/pull/11981>

- We have used the principles of the Java memory model to build a highly scalable thread-safety analyzer in CodeQL

